

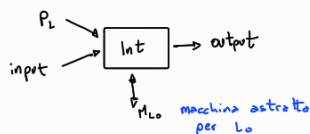
REALIZZAZIONE DELLA MACCHINA ASTRATTA



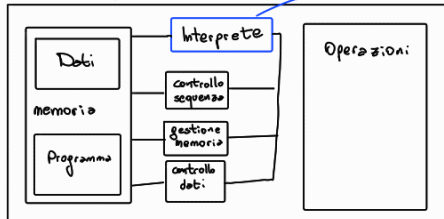
Interprete da L a L_0

$$I^{L_0}(P^L, \text{input}) = \text{output}$$

$$I^{L_0}: \text{Prog}^L \times D \rightarrow D$$



MACCHINA ASTRATTA

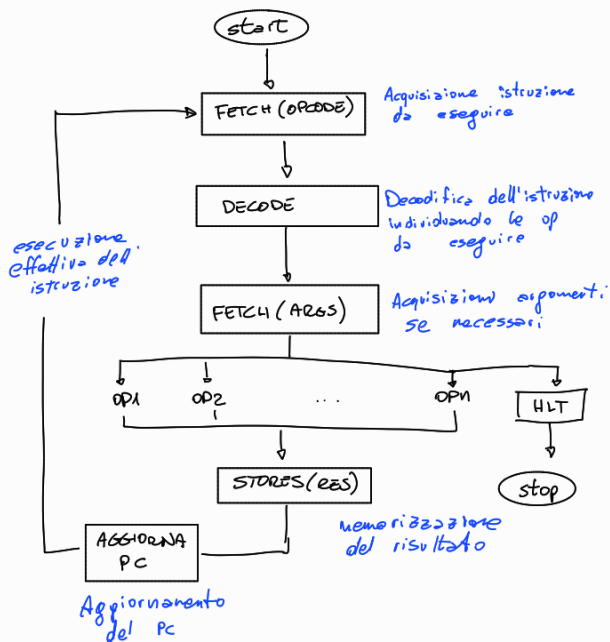


```

begin
  go := true;
  while go do begin
    FETCH (opcode, opinfo) at PC
    DECODE (opcode, opinfo)
    if opcode needs ARGS
    THEN FETCH (ARGS)
    case opcode of
      op1 : EXECUTE (op1, ARGS)
      op2 : ...
      ... : EXECUTE (opn, ARGS)
      HLT : go := false;
    if opcode has result
    THEN STORE (RES)
    PC := PC + size(opcode)
  end
end
    
```

controllo dati

controllo sequenza



SOLUZIONE COMPILATA

$\hookrightarrow$ Macchina astratta = compilatore

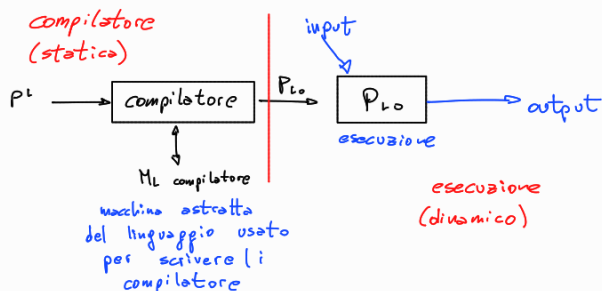
COMPILATORE: un compilatore da L a L_0

e un programma $C^{L_0}: \text{Prog}^L \rightarrow \text{Prog}^{L_0}$

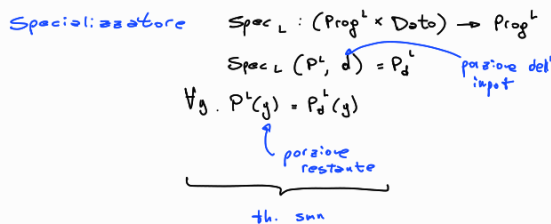
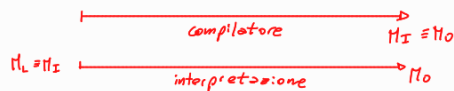
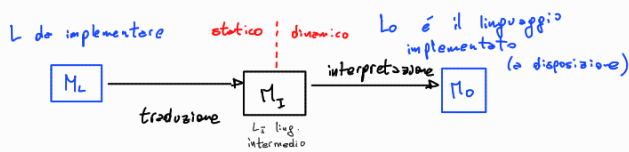
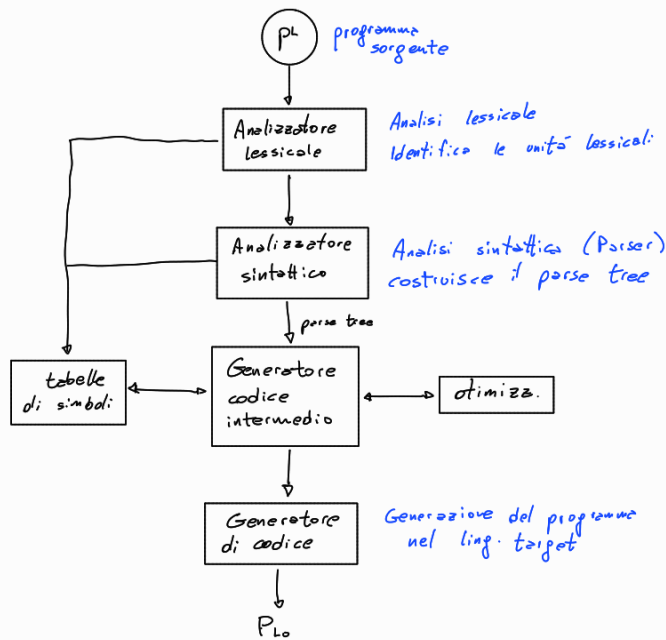
$$C^{L_0}(P^L) = P^{L_0} \quad \text{t.c.} \quad \forall d \in D$$

$$P^L(d) = P^{L_0}(d)$$

equivalenza
 P^L e P^{L_0} sono semanticamente equivalenti



STRUTTURA COMPILATORE



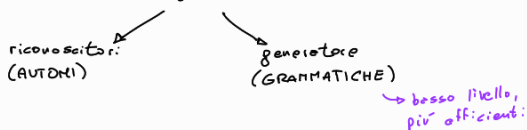
Specializzare un interprete su un programma
→ otteniamo un compilatore

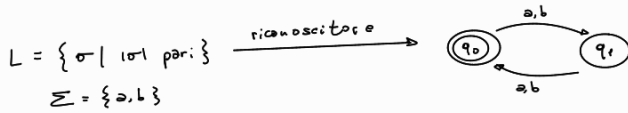
DESCRIVERE I PL

- ↳ **SINTASSI** → regole di costruzione di frasi ben formate
⇒ descrive relazioni tra simboli
- ↳ **SEMANTICA** → attribuzione del significato ai simboli
⇒ Data una frase che rispetta le regole sintattiche gli attribuisco un significato
- ↳ **PRAOTICA** → regole (non obbligatorie) di buon utilizzo (ingegneria del sw)
- ↳ **IMPLEMENTAZIONE** → come si eseguono frasi ben formate che hanno significato

Sintassi → descrive le regole di formazione

- parola stringa caratteri
- frase sequenza di parole ben formate
- linguaggio insieme delle frasi
- lessama parola con significato specifico (simboli terminali del programma)
- token categoria sintattica (simboli non terminali)





$G: S \rightarrow \varepsilon \mid a \circ S \mid a b S \mid b b S \mid b a S$

Descrizione della grammatica

- Alfabeto (simboli)
- Descrizione lessicale (sequenze di simboli che costituiscono parole del linguaggio)
 - "simboli" terminali
- Regole sintattiche (regole di derivazione della grammatica che compongono tra loro parole)
- Categorie sintattiche ("simboli" non terminali della grammatica)
 - token

=> Grammatiche CF (BNF)

CFG $G = \langle V, T, P, S \rangle$

V insieme finito di simboli non terminali:

elementi della frase (token)

categorie sintattiche

- Espressioni
- Costanti
- Dichiarativi
- Procedure

T insieme finito di simboli terminali:

vocabolario del linguaggio (lessami)

while, if, :=, {, } ...

+, -, ...

P produzioni: $A \rightarrow a \mid A \in V \quad a \in (V \cup T)^*$

regole di formazione del programma

con $\rightarrow$ while Exp do con ...

Exp ...

$S \in V$ simbolo iniziale categoria dei programmi

Esempio

Exp $\langle \{E\}, \{or, and, not, (,), 0, 1\}, P, E \rangle$

$E \rightarrow 0 \mid 1 \mid (E \text{ or } E) \mid (E \text{ and } E) \mid (not \ E)$